Day-5 Session

Quantum Computing with Qiskit Primitives & Noise Simulations with QSim, Qniverse

KEY SEMINAR MODULES

01

Qiskit Architecture

Foundational structures, transpiler pipelines, and execution flows.

02

Sampler & Estimator

Core primitives for probability distributions and expectation values.

03

Noise Models

Simulating environmental noise and device decoherence characteristics.

04

Introduction to IBM Quantum Platform

Accessing real hardware backends.

05

QSim & Qniverse

Advanced multi-platform classical simulators and noisy backend testing.

Presented by

C-DAC Hyderabad

Qiskit Architecture

Architecture Overview

Component	Functionality & Purpose
Qiskit SDK	The core Python framework to build and execute quantum programs.
Quantum Circuit	Representation of qubits, gates, measurements, and classical bits.
Transpiler	Converts high-level circuits for specific backend hardware execution.
Primitives	Interfaces (Sampler/Estimator) for measurement distributions and expectation values.
Backend/Runtime	Where the circuit actually runs—be it a simulator or quantum hardware.

The Qiskit SDK

The Qiskit SDK provides the comprehensive suite of tools required to develop and execute quantum applications from start to finish.

01
Create quantum circuits
Design and define the logic of your quantum algorithms.



02
Apply quantum gates
Manipulate qubits using single and multi-qubit operations.



03
Measure qubits
Extract classical data from quantum states into registers.



04
Transpile circuits
Optimize and map circuits to target specific physical hardware.



05
Run & process results
Execute on backends and analyze the output distribution.



Quantum Circuit
Qubits + Gates | Measurements

Qiskit Transpiler
Optimization | ISA Conversion

Qiskit Primitives
Sampler | Estimator

Backend / Runtime
Simulator | Quantum Hardware

Results

Quantum Circuit

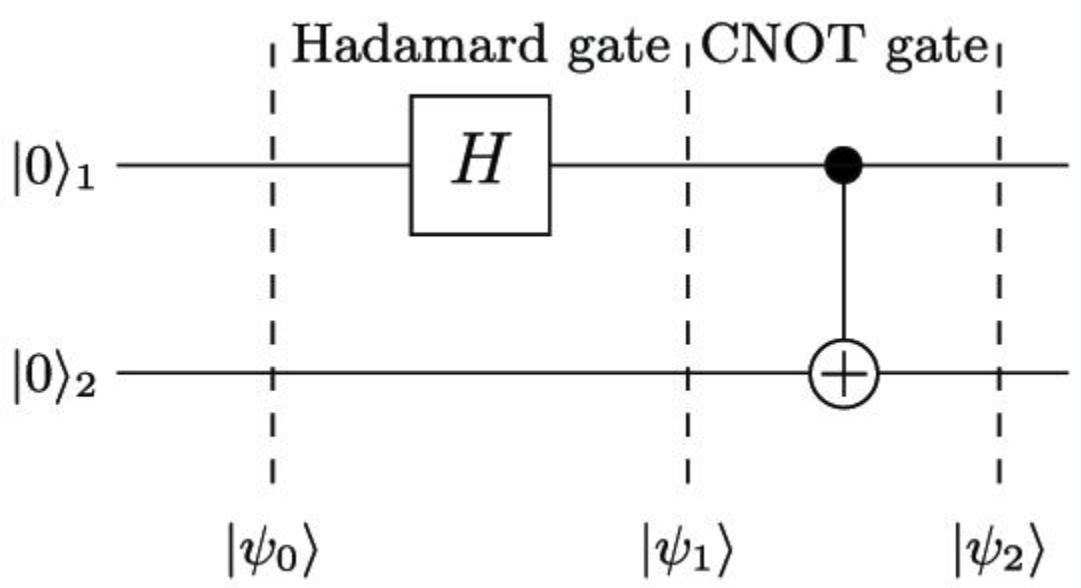
Fundamental Representation

The quantum circuit is the foundational representation of any quantum algorithm. It defines the logical sequence of operations before any hardware-specific translation occurs.

It contains qubits initialized to $|0\rangle$, unitary gates (like Hadamard or C-NOT), and measurement operations.

Circuit Example

A simple Bell-state preparation circuit:



The Qiskit Transpiler



Decomposition

Translates complex, high-level mathematical operations into the native, supported gate set of the specific target quantum processor.



Optimization

Reduces the overall circuit depth and gate count by canceling redundant operations, minimizing errors and decoherence during execution.



Routing

Maps logical qubits to physical hardware topology (SWAP insertion) and ensures compatibility with the **I**nstruction **S**et **A**rchitecture.

Hardware Constraints

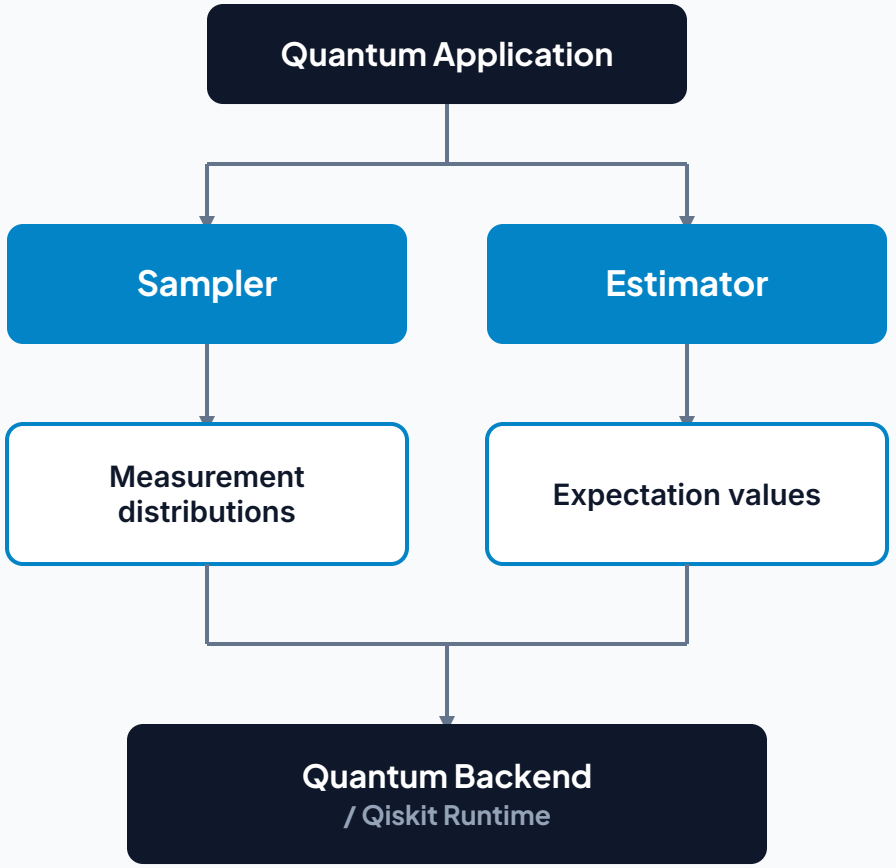
- **Supported Gate Set:** Quantum hardware can only physically execute a limited set of native gates. All abstract operations must be translated into this specific vocabulary.
- **Qubit Connectivity:** Two-qubit gates (like CNOT) can only be applied between physically connected qubits. The transpiler routes information to overcome limited topology.
- **Circuit Depth:** Qubits lose their quantum state over time (decoherence). The transpiler must minimize depth to execute the algorithm before the state collapses.
- **Hardware Noise:** Transpilation strategies account for error rates, preferentially placing critical operations on the highest-fidelity physical qubits available.

Primitives Interface

Bridging Application and Backend

Sampler and Estimator are not simply a layer that every circuit passes through after transpilation. They are advanced programming interfaces tailored for particular quantum workloads.

They sit between the application and the backend, abstracting the complexity of error mitigation and execution optimization, allowing developers to focus on the algorithmic output.



Sampler vs. Estimator

The Sampler

Used to obtain measurement outcomes and calculate their probability distributions. It outputs the quasi-probabilities of measuring specific bitstrings from a quantum circuit.

Ideal for: Search algorithms, cryptographic cracking, and sampling-based quantum machine learning.

The Estimator

Used to calculate and accurately estimate the expectation value of an observable (Hamiltonian) for the quantum state prepared by a given circuit.

Ideal for: Variational Quantum Eigensolver (VQE), QAOA, and simulating quantum chemistry models.

The Sampler Primitive

> After running this circuit, how frequently do I observe the outcomes 00, 01, 10, and 11?

MEASUREMENT DISTRIBUTIONS

The Estimator Primitive

> What is the expected value of this Hamiltonian for the quantum state produced by my circuit?

EXPECTATION VALUES

Execution Workflow

1. Application

Algorithm defined via Quantum Circuit with logical qubits and gates.

2. Transpiler

Optimizes and converts to Instruction Set Architecture (ISA).

3. Primitives

Sampler or Estimator interface chosen based on workload.

4. Backend

Execution on Simulator or QPU, returning final results.

Qiskit Primitives

Abstraction and Execution in Modern Quantum Algorithms

Evolution of Hardware Access

Raw Hardware Access

Developers directly handled hardware signals and quantum-control systems.

backend.run()

First abstraction. Allowed execution of circuits, returning raw 0s and 1s.

Qiskit Primitives

Higher-level interface designed for algorithm-specific outcomes.

The Need for Abstraction

> **Developers should focus on quantum algorithms, not the low-level details of controlling the QPU.**

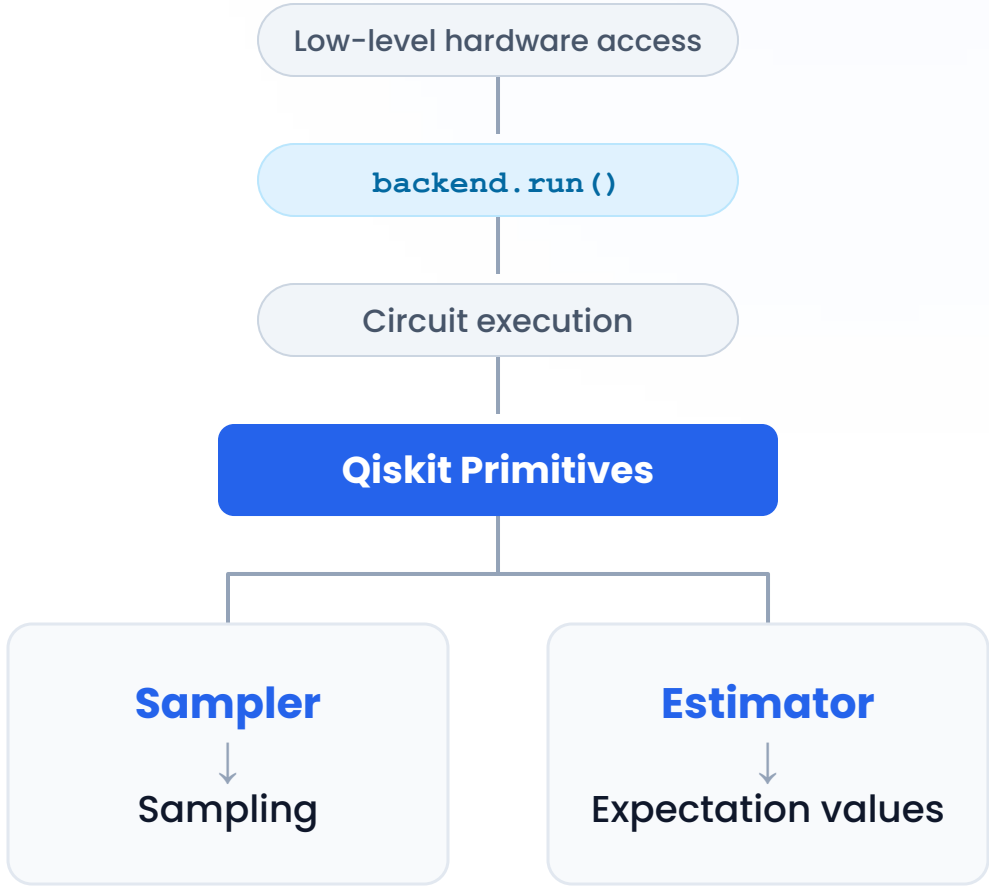
What Is a Primitive?

A primitive is a high-level building block providing a standard interface for specific quantum-computing tasks.

Abstraction Layer
Sits between the algorithm and quantum hardware.

Simplification
Avoids manual execution, result collection, and probability conversion.

Focus
Enables developers to request exact values like expectations directly.



The Two Main Qiskit Primitives

Sampler

Goal: Circuit Sampling

Executes a quantum circuit multiple times to measure the output distribution. It collects samples directly from the physical or simulated hardware to construct user-facing probability distributions.

Output

Bitstrings & Probability Distributions

Estimator

Goal: Expectation Values

Evaluates the expected value of an observable for a specific quantum state. It allows developers to calculate physical properties directly without needing to manually post-process raw measurement results.

Output

Expectation Values $\langle O \rangle$

SamplerV2

Core Question

"What classical outputs do I get when I execute this circuit?"

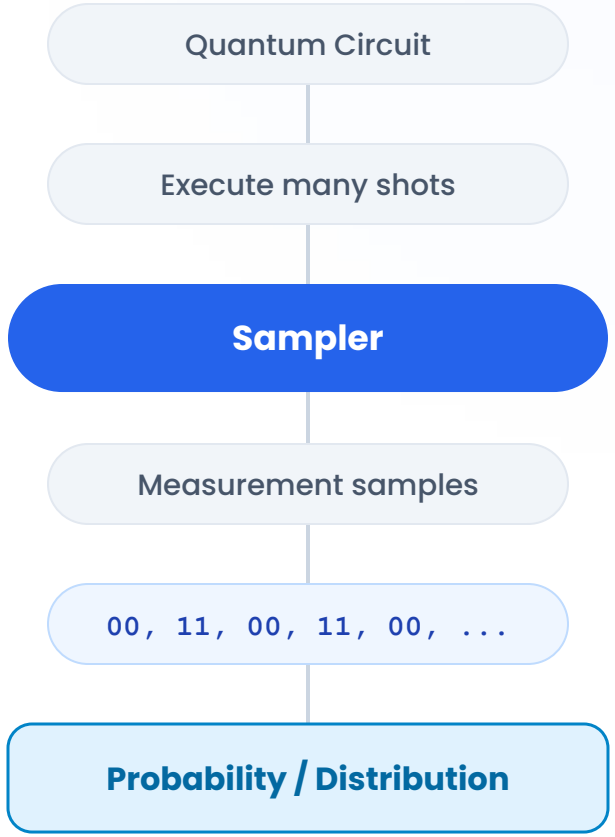
Example Application

Suppose a circuit produces a Bell state:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Outputs: 00 (≈50%), 11 (≈50%)

Key algorithms: QAOA, Grover's, Quantum Classification



EstimatorV2

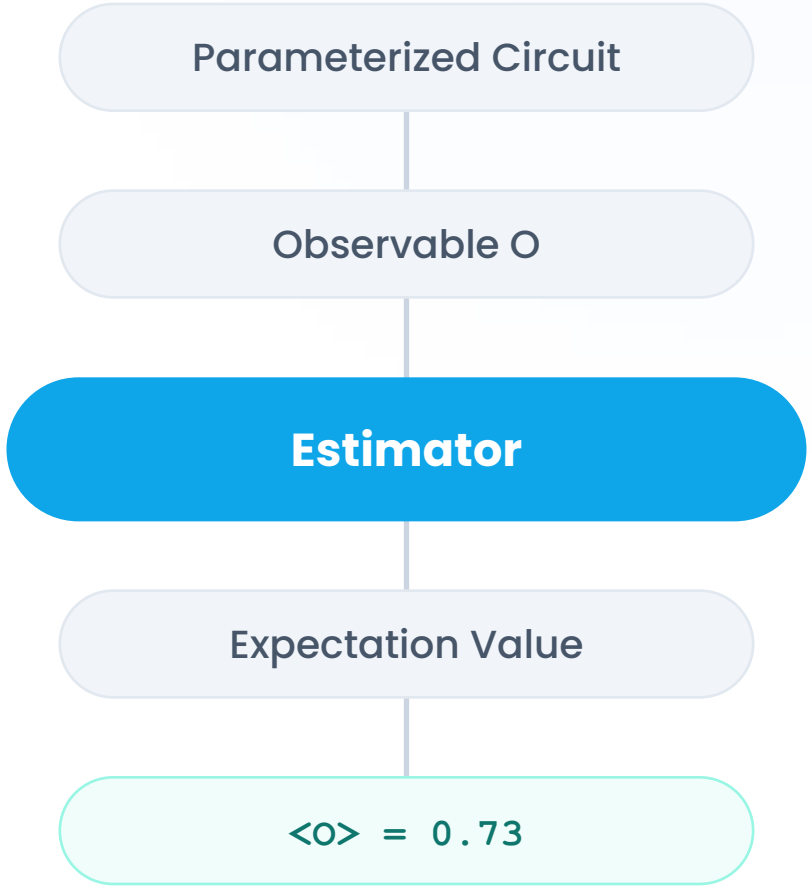
Core Question

"What is the expected value of an observable for the quantum state produced?"

$$\langle O \rangle = \langle \psi(\theta) | O | \psi(\theta) \rangle$$

Key Application

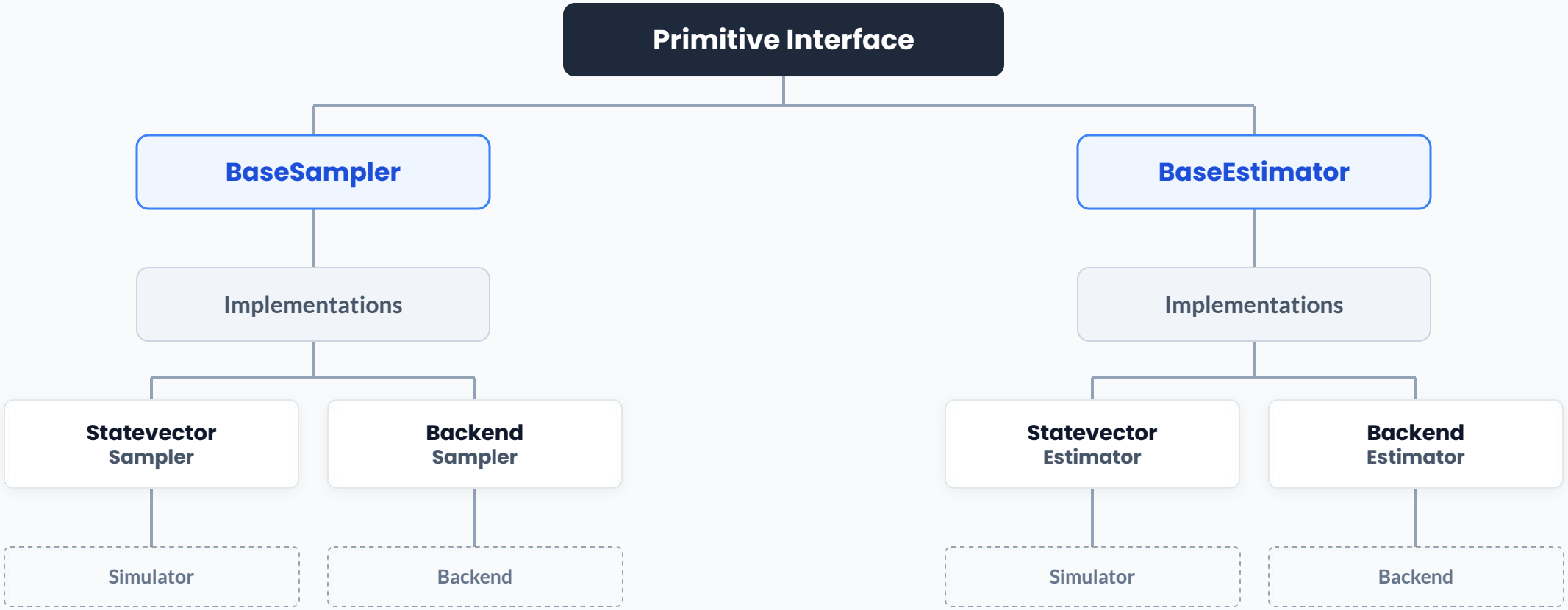
Provides the precise expectation value required for Variational Quantum Eigensolver (VQE) and chemistry simulations instead of raw measurement results.



Sampler vs Estimator

Feature	Sampler	Estimator
Main question	What outputs do I obtain?	What is the expectation value?
Input	Quantum circuit	Quantum circuit + observable
Output	Samples / measurement distribution	Expectation value
Example output	00: 49%, 11: 51%	$\langle H \rangle = -0.85$
Example algorithm	QAOA, Grover	VQE, QAOA

Primitive Interface Hierarchy



The Current Standard

V2

The Interface Standard

Future-Proofing Execution

Qiskit Primitives V2 is the current interface for developing modern quantum applications. While V1 remains usable, documentation and feature rollouts are focused entirely on V2.

V2 provides superior customization through detailed Options objects, specifically targeting shots, error mitigation, and hardware resilience.

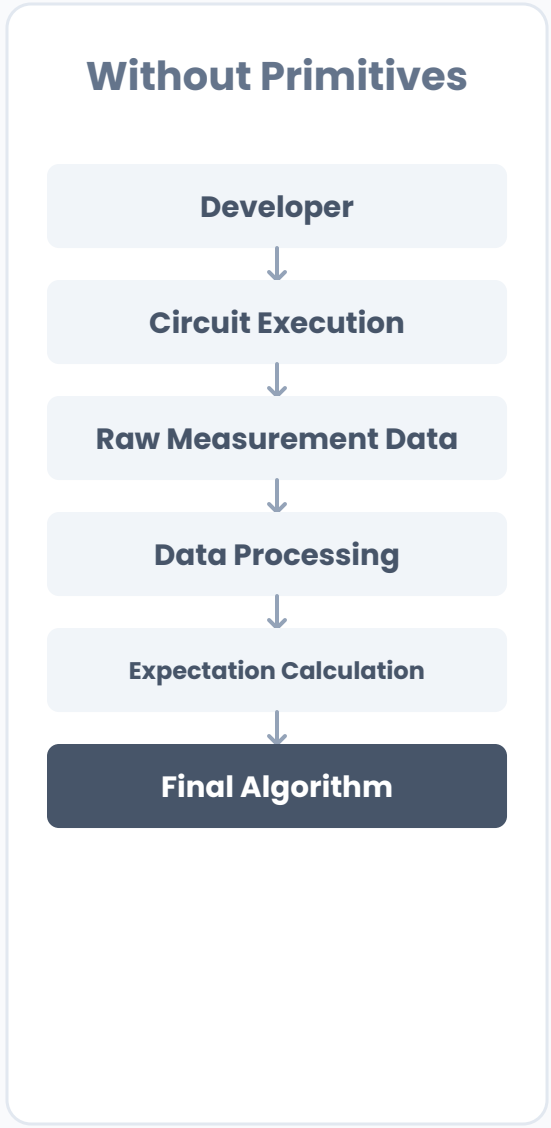
Why Are Primitives Important?

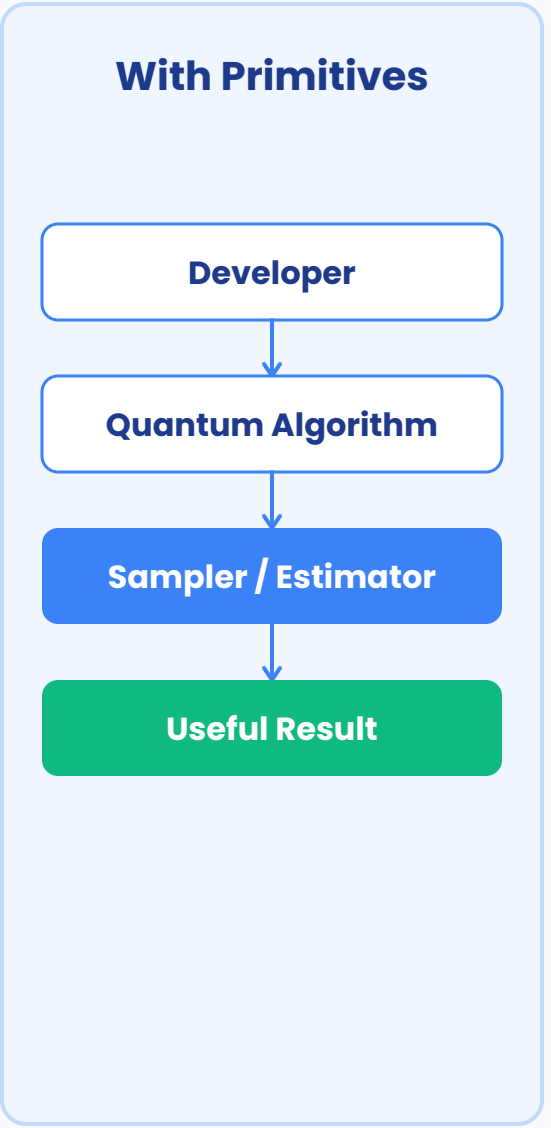
Abstract Complexity

Primitives abstract the complexity of quantum execution, allowing developers to concentrate purely on algorithmic design rather than data parsing.

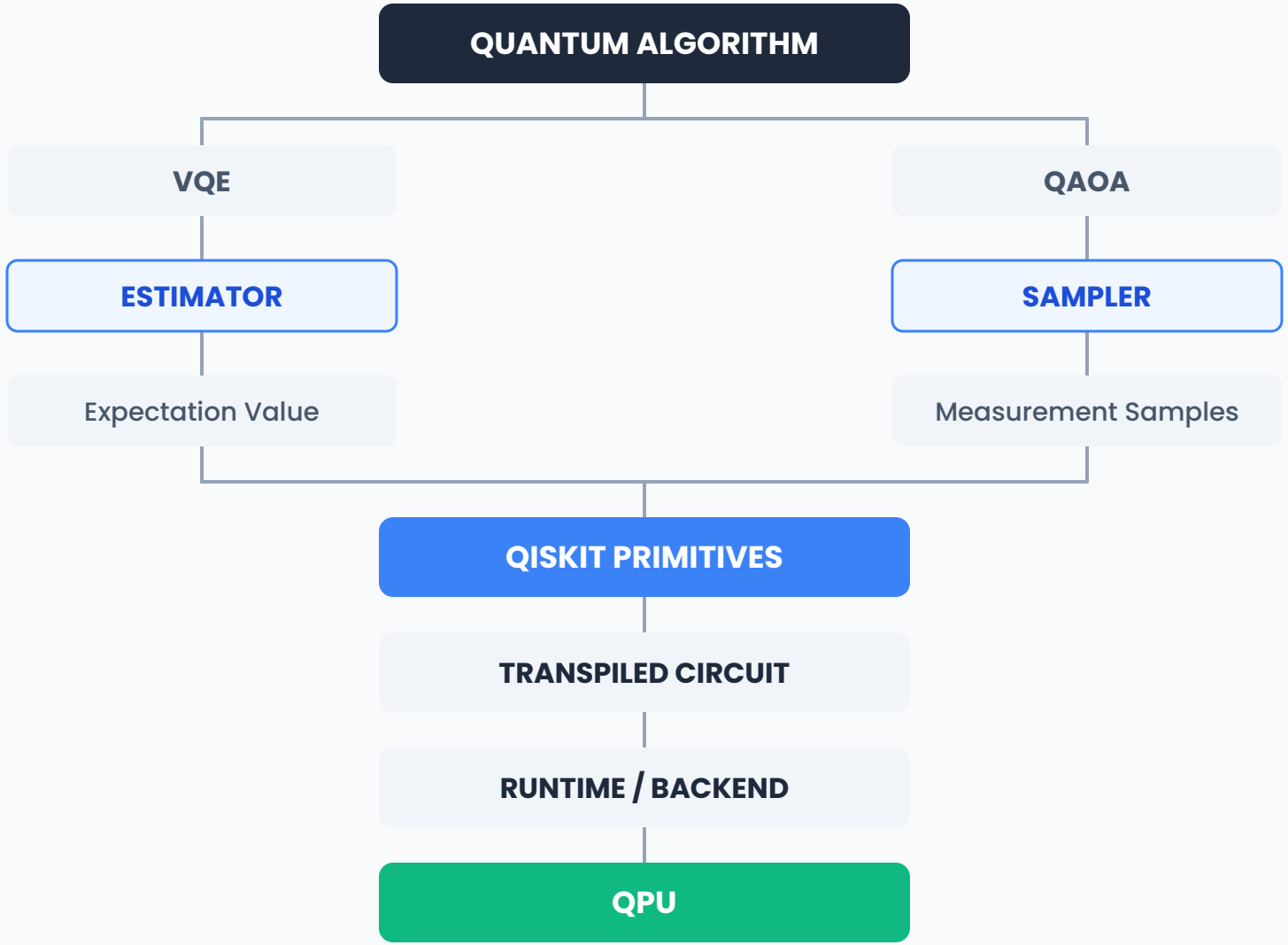
Native Hardware Resilience

By leveraging hardware-specific implementations, primitives natively inject state-of-the-art error mitigation and resilience directly into the execution pipeline, completely transparent to the end user.



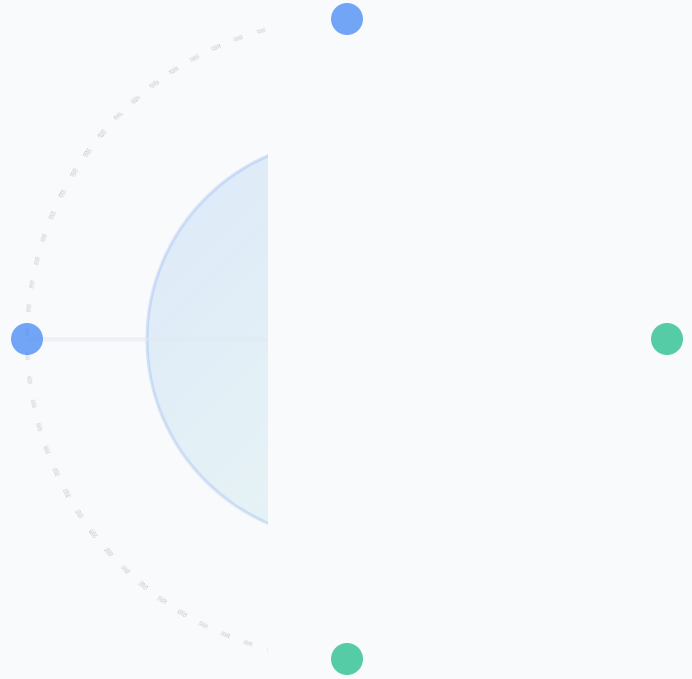


Algorithm Execution Flow



Introduction to IBM Quantum Platform

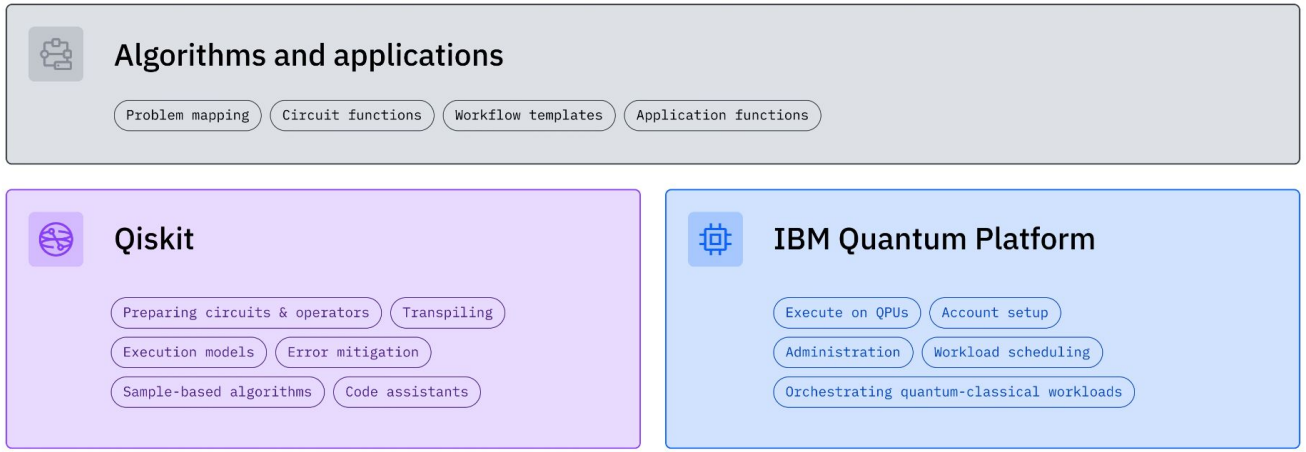
Explore the enterprise-grade ecosystem for running quantum workflows at scale.



IBM Quantum

Platform Ecosystem

A comprehensive toolkit and enterprise-grade cloud platform for building, optimizing, and running quantum workflows at scale.



Qiskit Framework

Modular, extensible toolkit for quantum algorithms, HPC, and quantum information science.

Workflow Capabilities

Build, optimize, and execute quantum circuits with highly specialized tools.

IBM Quantum Platform

Access real quantum computers via Quantum Compute Service and Qiskit Functions Catalog.

Error Mitigation

Advanced techniques for noise suppression and primitives for reliable hardware execution.

Ecosystem

Open-source projects extend Qiskit's core functionality and integrate seamlessly with outer systems.

Support & Community

Error code registry, detailed FAQs, and rich collaboration resources for troubleshooting.

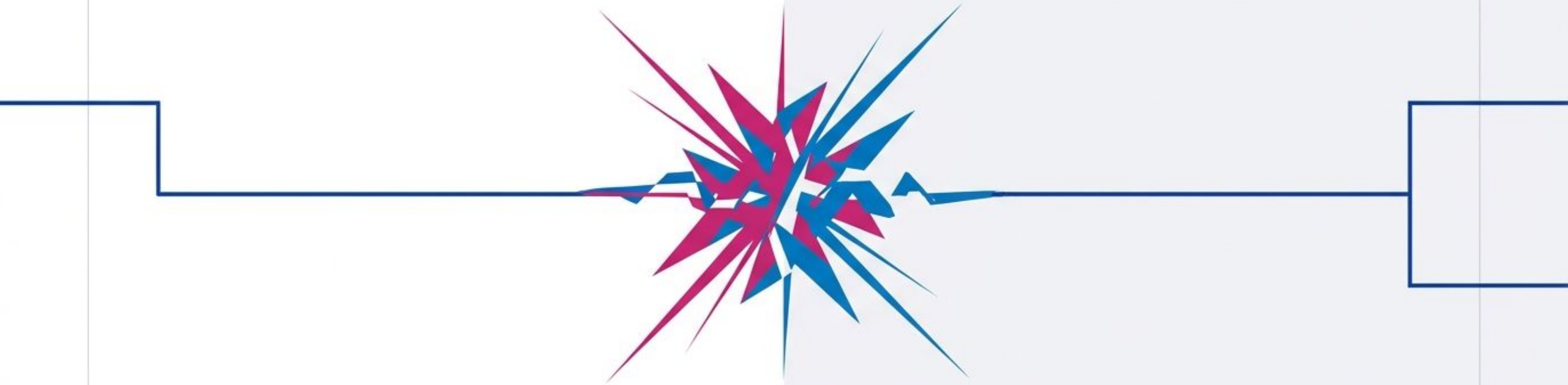
Learning Resources

Access structured tutorials, professional courses, and curated learning paths designed specifically for skill development in quantum computing.

Source: https://quantum.cloud.ibm.com/

Simulating Quantum Reality

Building custom noise models with Qiskit Aer.



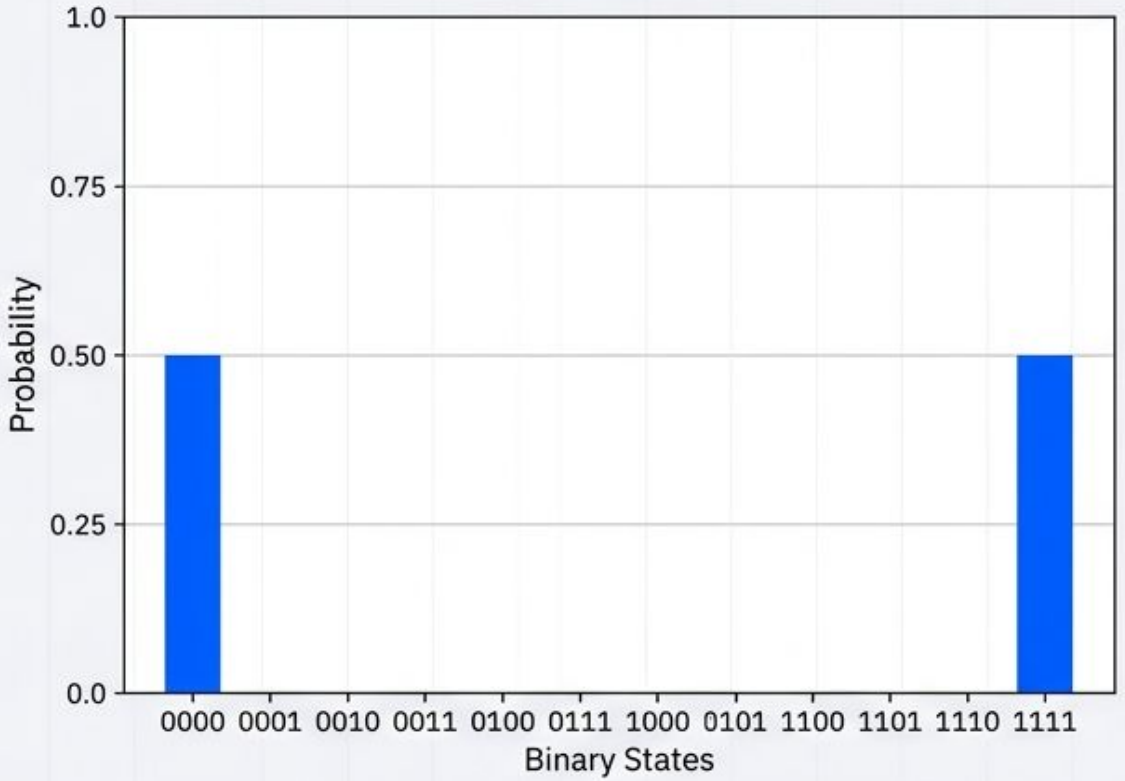
Designed for qiskit[all]]~=2.5.1, qiskit-ibm-runtime~=0.47.0, and qiskit-aer~=0.17.

Source: https://quantum.cloud.ibm.com/docs/en/guides/build-noise-models#noise-model-examples

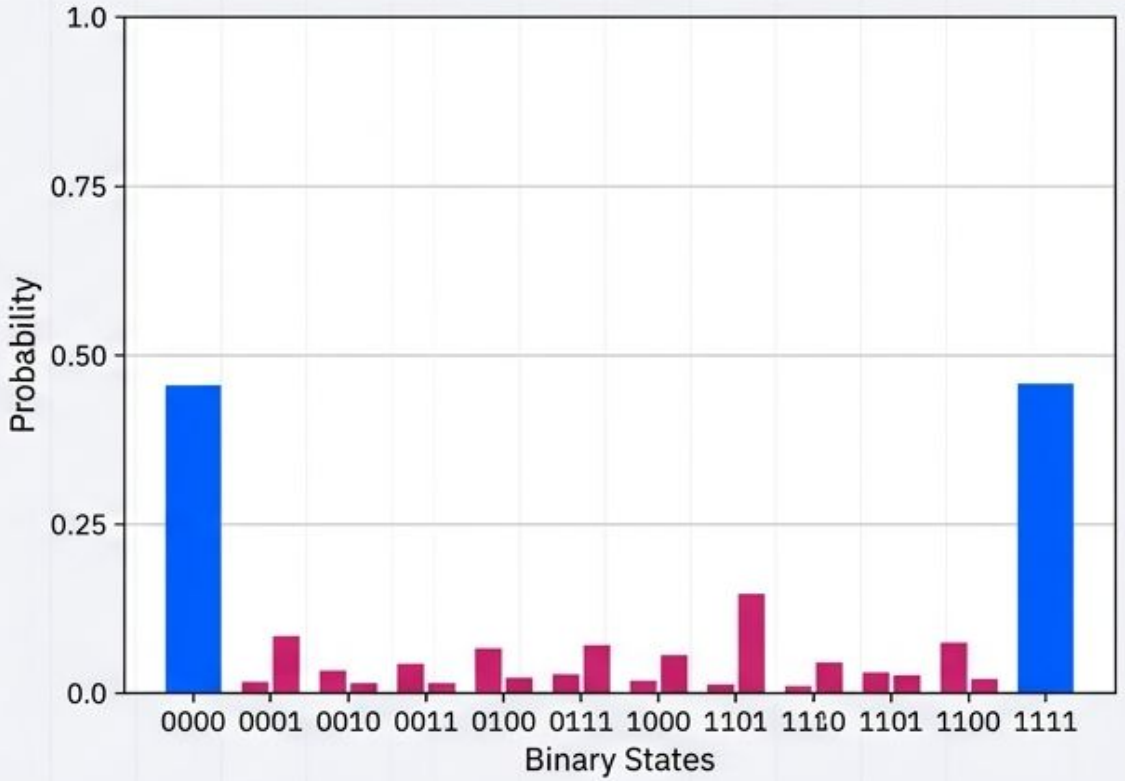
The Gap Between Theory and Hardware

Algorithms designed in perfect mathematical isolation degrade when executed on physical quantum processors. Emulating these physical imperfections is a prerequisite for studying error impacts and developing mitigation strategies.

The Ideal



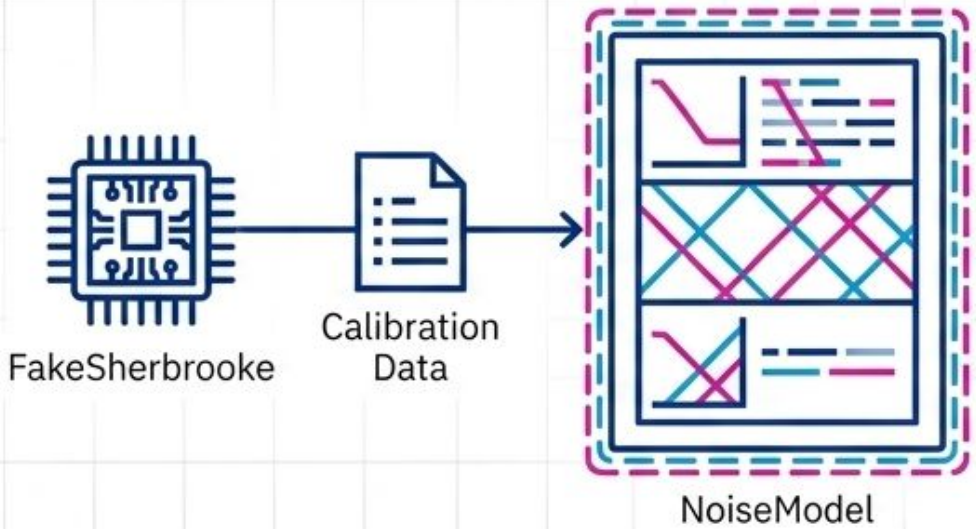
The Reality



The Two-Line Physical Clone

Before building custom environments, you can automatically extract and apply the latest calibration parameters from any Qiskit-compatible real or fake backend.

```
from qiskit_ibm_runtime.fake_provider import FakeSherbrooke
backend = FakeSherbrooke()
noise_model = NoiseModel.from_backend(backend)
```

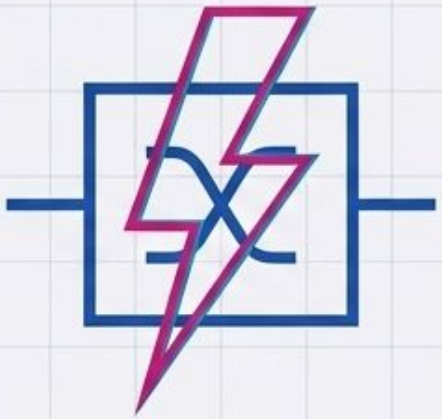


The Qiskit Aer Noise Architecture



NoiseModel

The container. Stores the complete noise environment used by the AerSimulator.



QuantumError

Describes CPTP (Completely Positive Trace-Preserving) gate errors. Applied after gate/reset instructions, or before measure instructions.



ReadoutError

Describes classical measurement errors (assigning the wrong bit value).

The Diagnostic Matrix of Quantum Errors

General & Unitary Mechanics

`kraus_error`: General n-qubit CPTP error channel (list of Kraus matrices).
`mixed_unitary_error`: n-qubit mixed unitary error (unitaries + probabilities).
`coherent_unitary_error`: n-qubit coherent unitary (single matrix U).

Stochastic & Pauli Mechanics

`pauli_error`: Mixed unitary list of Paulis and probabilities (e.g., bit-flip, phase-flip).
`depolarizing_error`: n-qubit depolarizing channel parameterized by probability p.
`reset_error`: Single-qubit reset parameterized by probabilities p0, p1 to states $|0\rangle$, $|1\rangle$.

Damping & Thermodynamic Mechanics

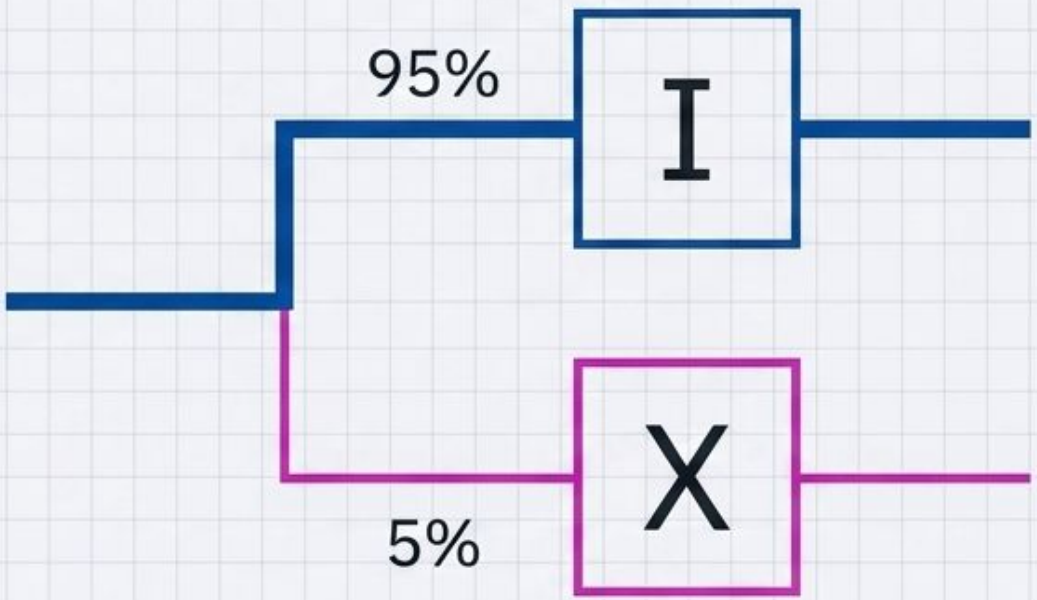
`thermal_relaxation_error`: Parameterized by T1, T2, gate time t, and thermal population.
`amplitude_damping_error`: Parameterized by amplitude parameter λ.
`phase_damping_error`: Parameterized by phase parameter γ.
`phase_amplitude_damping_error`: Combined generalized damping channel.

Constructing Fundamental Pauli Errors

Instantiating probability-weighted error channels.

```
p_error = 0.05
# 5% chance of X (bit-flip), 95% chance of Identity
bit_flip = pauli_error([("X", p_error), ("I", 1 - p_error)])

# 5% chance of Z (phase-flip), 95% chance of Identity
phase_flip = pauli_error([("Z", p_error), ("I", 1 - p_error)])
```



Combining Error Operators

Composition (error1.compose(error2))

bit_flip.compose(phase_flip) creates a combined XZ error.

$E(\rho) = E_2(E_1(\rho))$



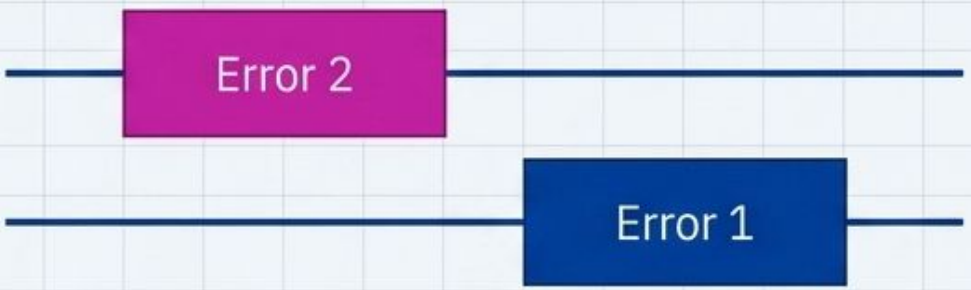
Tensor Product (error1.tensor(error2))

$E(\rho) = (E_1 \otimes E_2)(\rho)$



Expand Product (error1.expand(error2))

$E(\rho) = (E_2 \otimes E_1)(\rho)$

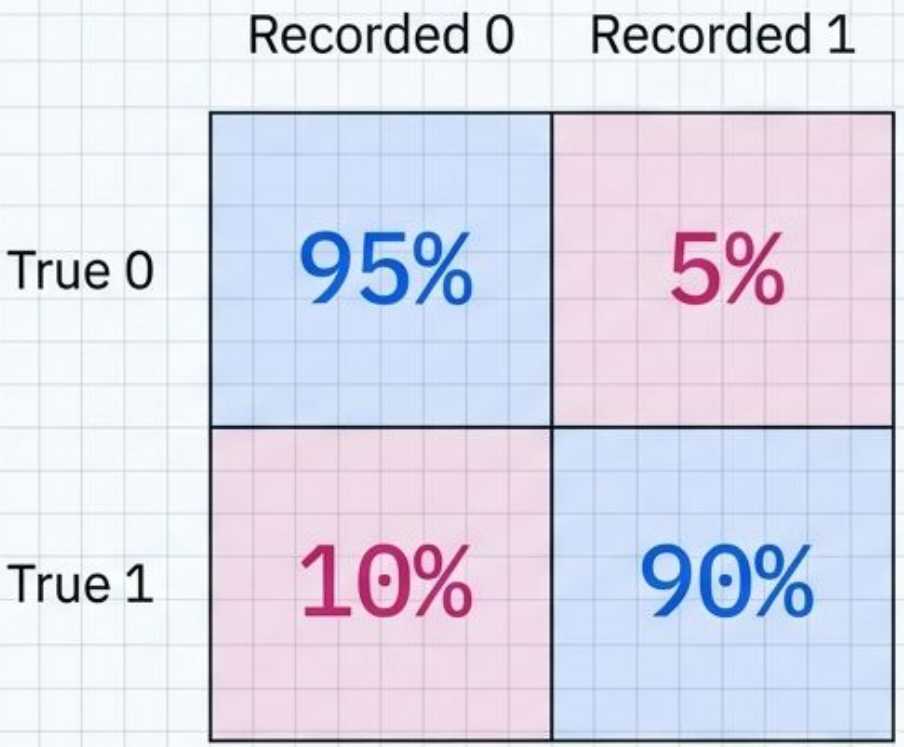


Readout Errors at the Measurement Layer

Specifying classical misassignment vectors P(A|B), where A is the recorded classical bit and B is the true quantum state.

```
p0given1 = 0.1   # 10% chance to record 0 when true state is 1
p1given0 = 0.05  # 5% chance to record 1 when true state is 0

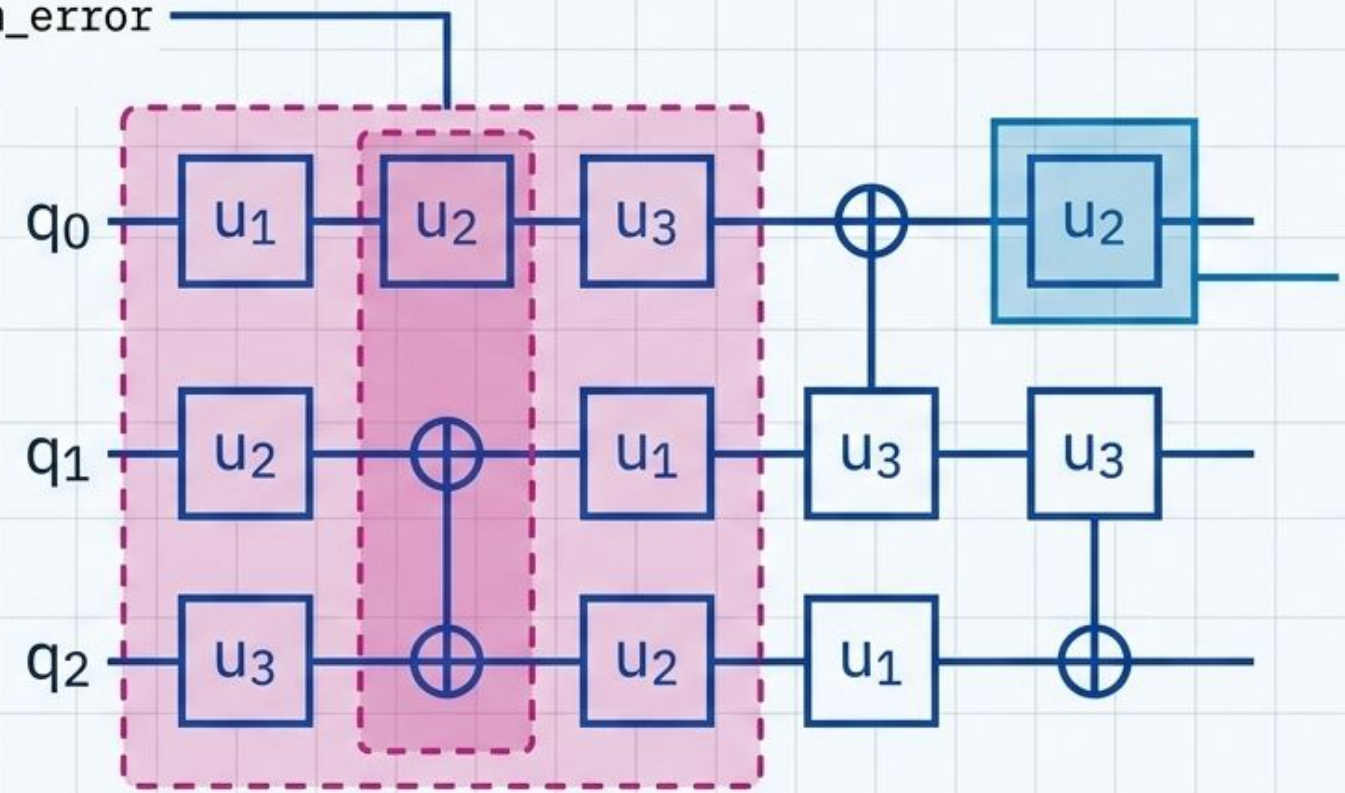
ReadoutError([[1 - p1given0, p1given0],
              [p0given1, 1 - p0given1]])
```



The Scope of Error Targeting

Method 1:

`add_all_qubit_quantum_error (error, instructions)`
Applies universally to any occurrence of an instruction.



Method 2:

`add_quantum_error (error, instructions, qubits)`
Targets specific instructions on specific qubits (Note: Qubit order matters for multi-qubit gates).

Modeling Physical Thermodynamics

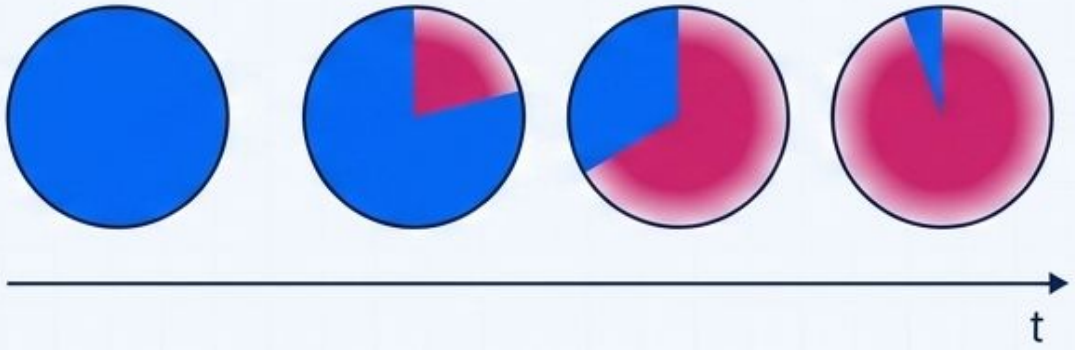
Creating a thermal relaxation model based on qubit environment interactions, bounded by the physical constraint $T_2 \leq 2T_1$.

T1 (Relaxation Time): Sampled normal distribution (mean: 50µs).

T2 (Dephasing Time): Sampled normal distribution (mean: 70µs, truncated to 2T1).

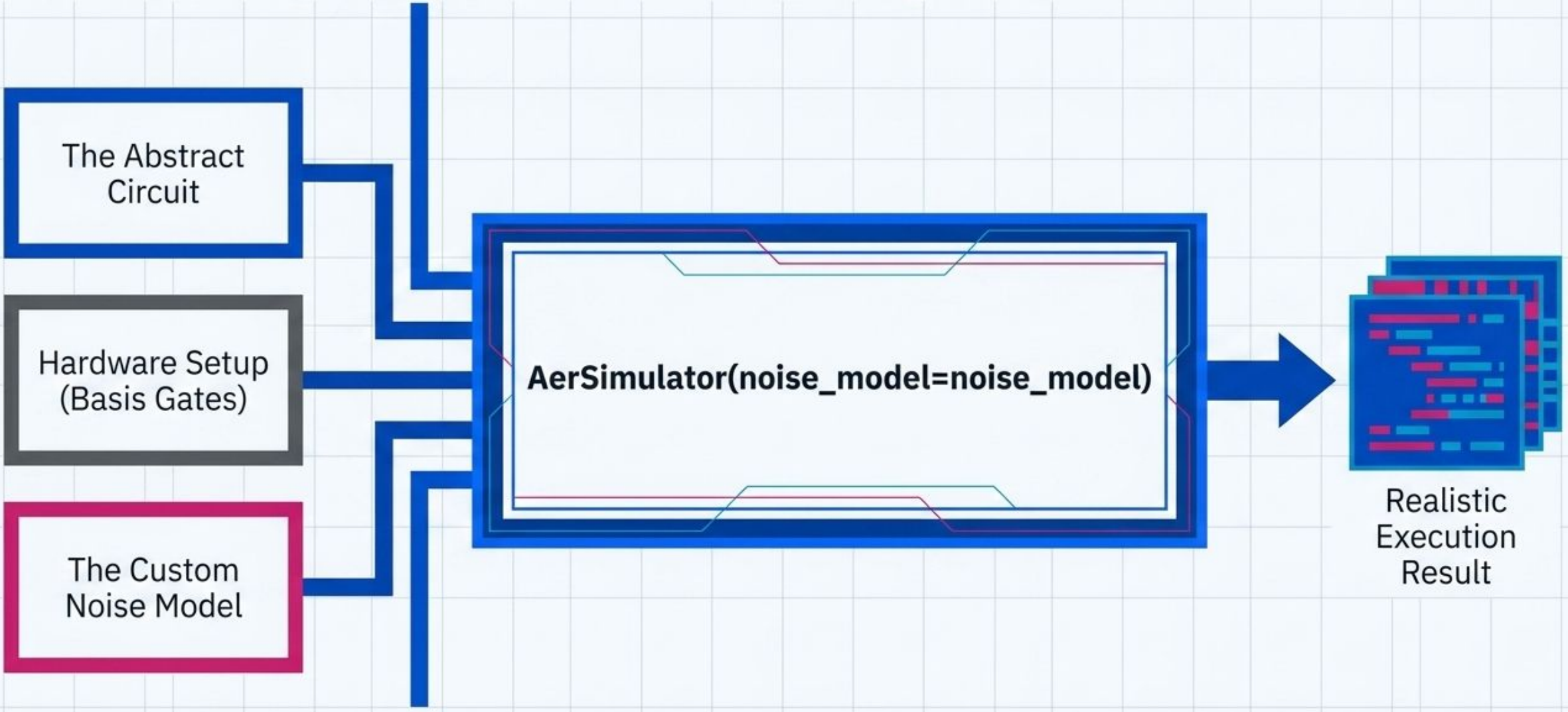
Gate Times (ns): u1=0, u2=50, u3=100, cx=300, measure/reset=1000.

```
# Constructing the instruction error
errors_u2 = [
    thermal_relaxation_error(t1, t2, time_u2)
    for t1, t2 in zip(T1s, T2s)
]
```



Gemini Notebook

The Complete Simulation Pipeline



Gemini Notebook

Noise Simulations with QSim and Qniverse

An introductory overview to simulating noise-affected quantum systems and computing toolkits

Getting Access to QSim & Qniverse

Follow these steps to register and gain access to the quantum computing toolkits.

science

QSim Toolkit

Quantum Computer Simulator Toolkit

QSim
OC Workbench

- **Official Website**
  https://qctoolkit.in/
- **Request Access**
  Send an email to:
  qcworkbench@cdac.in

इलेक्ट्रॉनिकी एवं सूचना प्रौद्योगिकी मंत्रालय
MINISTRY OF ELECTRONICS AND INFORMATION TECHNOLOGY
सत्यमेव जयते

Qniverse

Qniverse Platform

Explore quantum simulations and community resources

- **Official Website**
  https://qniverse.in
- **Registration Link**
  https://qniverse.in/register/
- **YouTube Channel**

https://www.youtube.com/@qniversedotin

Introduction to Qniverse

A national computing platform designed to accelerate quantum innovation and collaboration across India

About Qniverse

National platform connecting quantum researchers, students & enterprises across India

Developed by Quantum Technology Group, CDAC Bangalore

Designed to accelerate quantum innovation and collaboration

Quantum Ecosystem

Integrated environment to design, simulate & execute quantum algorithms

Supported Technologies:

- Superconducting
- Photonic
- Neutral Atom

HPC Acceleration

Integrated with High-Performance Computing systems

Simulation acceleration:

- CPUs & GPUs
- FPGAs
- Vector Processors

Supported Simulators

Pre-integrated industry-standard simulation frameworks:

⚡**Qiskit**
⚡**CirQ**
⚡**CudaQ**
⚡**QuEst & Qulacs**

Key Capabilities & Features

A comprehensive quantum platform built for end-to-end circuit design, optimization, and physical execution

Circuit Builder

Powerful drag-and-drop environment for quantum gate design.

Features:

- Drag-and-drop design
- Rich gate library
- Real-time simulation
- Active error checking

Accelerator Simulation

Simulate highly complex circuits on hybrid architecture platforms.

Supports:

- High-speed GPUs
- Custom FPGAs
- Vector processors
- Large-scale scaling

Real Quantum Hardware

Run algorithms on actual quantum physical processors directly.

Integrations:

- IBM Quantum hardware
- Physical system execution
- Direct cloud access
- Live telemetry

Integrated Transpiler

Optimize high-level abstract circuits for target hardware.

Converts:

- QASM source code
- Qiskit programs
- Hardware optimization
- Gate depth reduction

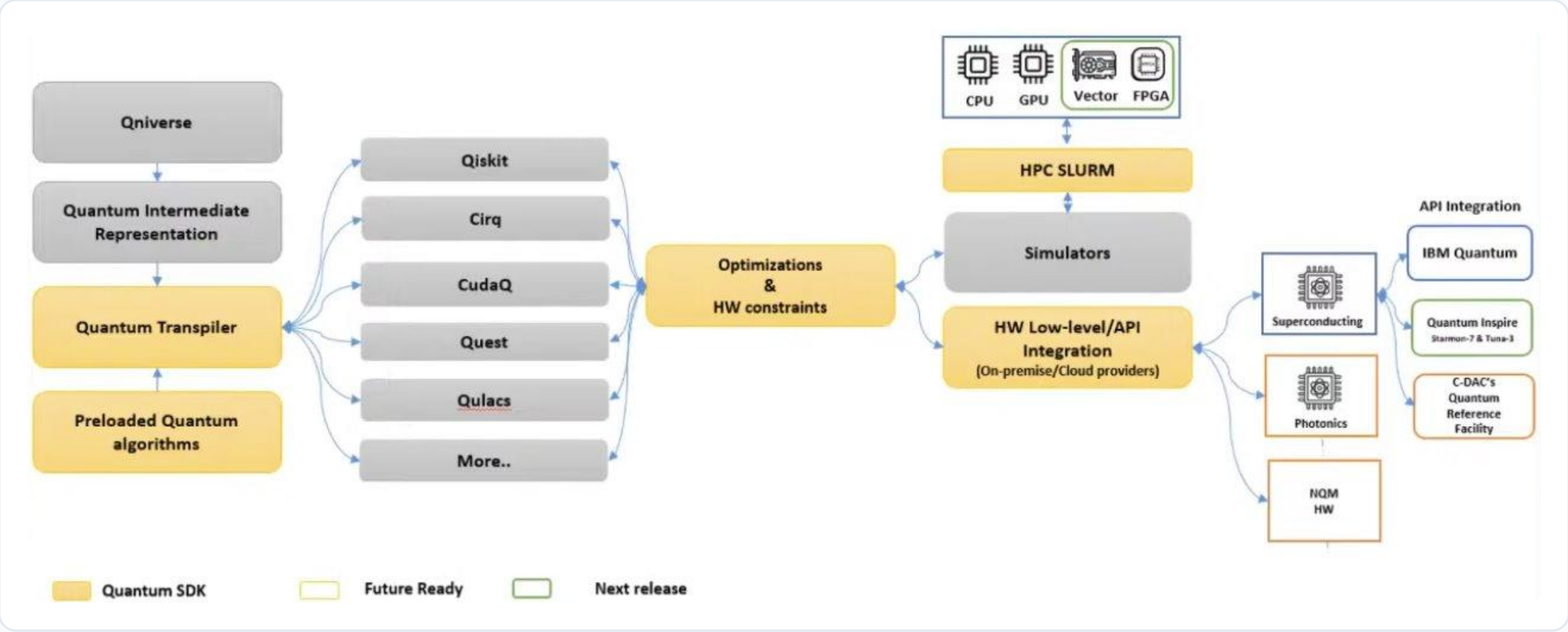
Algorithms & Libraries

Bridges theoretical quantum mechanics to practical execution.

Components:

- Modular libraries
- Standard algorithms
- Efficient templates
- Rapid prototyping

Architecture



Custom Noise Parameters

Configure and model physical qubit and gate imperfections within the simulation environment

Initialization Error

State preparation flaws before computation begins.

- Occurs when qubit fails to initialize correctly (expected $|0\rangle$ flips to $|1\rangle$).
- Introduces unintended states early.

Readout Error

Measurement-specific errors during active readout.

- Recorded state is incorrect (e.g., $|0\rangle$ read as $|1\rangle$).
- Misrepresents the final computation outcomes.

Gate Rotation Error

Imperfect physical calibration of quantum gates.

- Deviations (εx, εy, εz) from target Rx, Ry, Rz rotation angles.
- Transpiled and injected directly into basis circuits.

Depolarization Error

Environmental noise leading to state mixing.

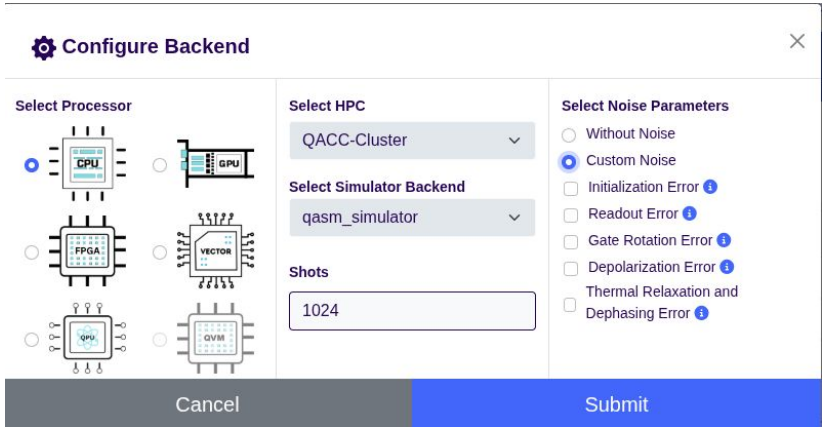
- Random flips (X), phase-flips (Z), or both (Y) with equal probability.
- Leaves target qubits partially or fully mixed.

Thermal & Dephasing

Time-dependent environmental relaxation.

- Qubits lose energy (relax to $|0\rangle$) or coherence (dephase).
- Controlled by parameters T1 (relaxation) and T2 (dephasing).

settings Backend Configuration



Thank you!